XSS (Cross-Site Scripting) Report

This technical report documents the authorized testing of a reflected Cross-Site Scripting vulnerability in the PortSwigger lab environment.

---

1. Lab Name

PortSwigger: Reflected XSS into HTML context with nothing encoded

2. Objective

To exploit a reflected XSS vulnerability in a search function to execute JavaScript code in the victim's browser.

---

3. Vulnerability and Concept

Cross-Site Scripting (XSS) occurs when user input is reflected back to a page without proper encoding. Reflected XSS occurs when the malicious input is included in an immediate server response and is then interpreted by the victim's browser.

Reflected XSS sequence

1. User input is sent to the server.
2. The server reflects the input in its response.
3. The browser interprets and executes the input as code.

The search parameter in this lab is vulnerable. The exact test payload is `<script>alert(1)</script>`. The server reflects the payload without encoding, causing the browser to execute it.

---

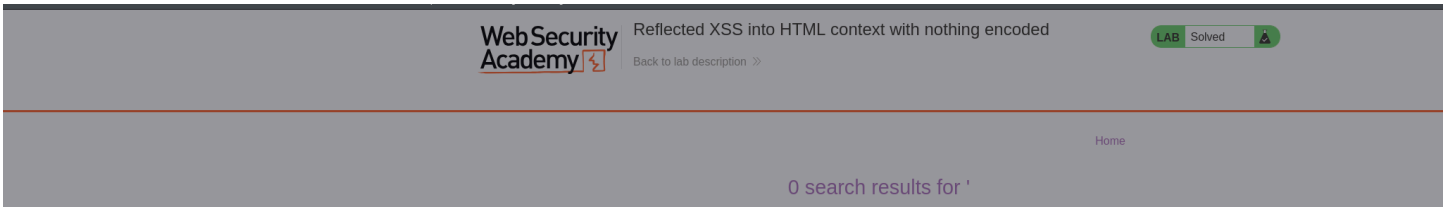
4. Testing Approach

1. Navigated to the shopping website.
2. Found the search bar.
3. Entered the test payload: `<script>alert(1)</script>`.
4. Pressed Enter.
5. The script executed and an alert box appeared.
6. The lab was solved.

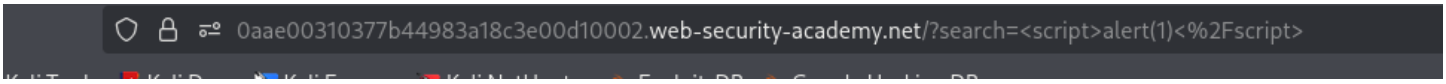
---

5. Evidence and Screenshots

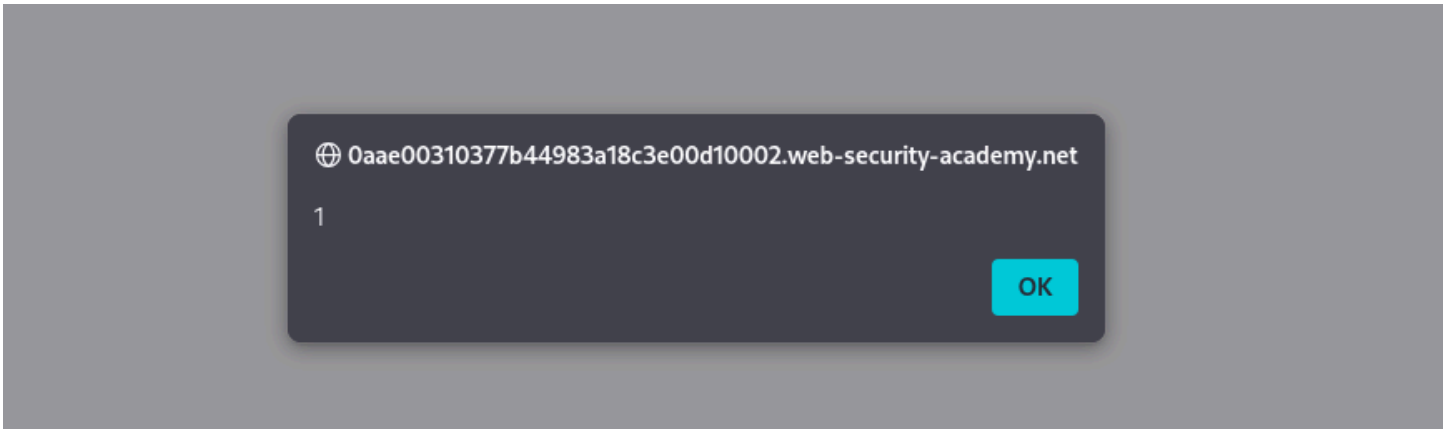
1. Lab completion, showing the “Solved” banner.



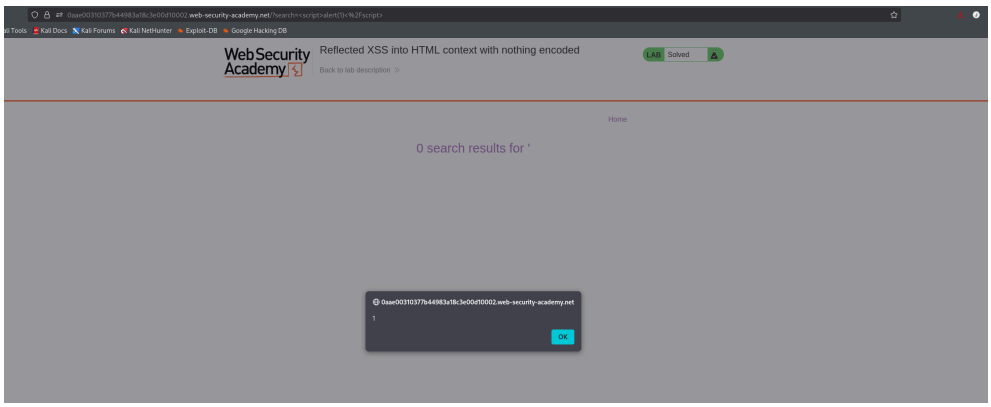
2. URL containing the XSS payload.



3. Alert box showing successful JavaScript execution.



4. Full-page evidence.



---

6. Impact

The potential impact of an XSS vulnerability includes:

- **Theft of user cookies and sessions** — An attacker may access sensitive authentication information when protections are insufficient.
- **Performing actions as the user** — Malicious scripts may act within the user's active session.
- **Website defacement** — An attacker may alter the content displayed to users.
- **Redirecting users to malicious sites** — Victims may be sent to attacker-controlled or harmful destinations.
- **Keylogging and data theft** — Malicious scripts may capture keystrokes or other accessible data.

---

7. Remediation

- **Output encoding** — Encode output for its intended context, especially by applying HTML entity encoding when inserting text into HTML.
- **Input validation** — Validate input against the expected format and reject unexpected values where appropriate; this should supplement, not replace, output encoding.
- **Content Security Policy (CSP)** — Deploy a restrictive CSP to reduce the ability of injected scripts to execute and limit the potential impact of XSS.
- **Frameworks that automatically escape output** — Use framework features that safely escape dynamic content by default, while reviewing any explicitly unsafe rendering functions.
- **HttpOnly cookies** — Mark sensitive cookies as HttpOnly to prevent client-side JavaScript from reading them, reducing cookie theft through XSS.

---

8. What Was Learned

- XSS occurs when input is not encoded.
- Reflected XSS originates from an immediate server response.
- JavaScript can execute in a victim's browser.
- Output encoding helps prevent XSS.
- Even a simple alert demonstrates that the vulnerability is exploitable.

---

9. Conclusion

The testing was performed in the authorized PortSwigger lab environment. Reflected XSS should only be tested with permission and within an approved scope.
